Octria — Planning Document

Nov 3, 2026 · @Abhin

Overview

A free, cross-platform app that turns any task into a classroom with an AI-generated roadmap of phases and steps, shown as an interactive node graph, for one person or a group.

- **Who it's for:** students, teams and individuals running anything from an app build to exam prep, research, events or creative work.
- **Hero feature:** the node graph. It decides whether people keep using the app, so it gets the most design and build time.
- **Principles:**
  - Free to use, so every design choice keeps running costs near zero.
  - One codebase for Android, iOS, web and desktop.
  - AI drafts, humans decide: every generated plan, split and skill list is editable.
  - Easy first: a new user should understand their next step within seconds of opening a classroom.

Tech stack

Flutter plus Firebase covers every platform from one codebase, with Gemini's free tier for all AI features.

Layer	Choice	Why
App	Flutter (Android, iOS, web, Windows, macOS, Linux)	One codebase; you already know it; custom canvas drawing for the graph
Sign-in	Firebase Auth: Google, GitHub, email	Google is required; GitHub doubles as the repo connection
Database	Cloud Firestore	Real-time sync, offline cache, security rules for permissions
Server logic	Cloud Functions	Keeps AI and GitHub keys off the device; runs scheduled jobs
Files	Google Drive API (`drive.file` scope)	Files live in users' own Drive, so storage costs you nothing

Layer	Choice	Why
Notifications	Firebase Cloud Messaging	Free push on all platforms
AI	Gemini API (free tier)	See below

AI provider: Gemini

Decided: Gemini API free tier for every AI feature. Claude Pro was ruled out because it covers the Claude apps only, not API calls.

- The free tier has rate limits, so results are cached and each user gets a daily AI call limit.
- Free-tier prompts may be used by Google to improve its products; check the current terms before sending resumes through it.
- All AI calls go through one provider layer (`generatePlan` , `extractSkills` , `splitWork`), so a paid Gemini tier or another model can be swapped in later if needed.

Features

Everything below is in scope; the Build phases section says when each arrives.

Account and profile

- Sign in with Google (plus GitHub and email).
- Profile page: name, photo, bio, skills, resume, linked GitHub, editable at any time.
- Upload a resume; AI extracts skills; the user confirms, edits or adds more by hand.
- Growth map: skills and experience over time (see Tracking and growth).

Classrooms

- Create a task, then choose Individual or Group.
- A classroom is created for the task; the creator is the master.
- Invite members by code or link; join requests can be approved by the master.
- Groups of up to 10 members, with a co-master role; the co-master inherits the classroom if the master leaves.
- Archive a finished classroom; save its plan as a reusable template.

AI planning

- AI reads the task, asks short follow-ups (deadline, hours per week, goal, level) and classifies the task type.
- Generates phases, steps within each phase, dependencies, time estimates and a timetable.
- For groups, splits steps by member skills and availability; the master approves or edits.
- Re-plan on demand: a member drops out, a deadline moves, or work falls behind.
- In-classroom AI assistant: "What should I do next?", "Explain this step".

Phases and steps

- Phases are the major sections; steps are the individual pieces of work inside them.
- Created in suggested order, but any step can be worked on in any order.
- Dependencies between steps, including across members, shown on the graph.
- Soft lock: starting a step whose dependency isn't done shows a warning, not a block.
- Mark done: a member marks a step done; the master can approve it or send it back. Approval can be turned on or off per classroom.
- Each step holds its own files, links, videos, notes and comments.

Resources

- Shared classroom library and a private library per member.
- Any resource type: files, links, YouTube or other video, images, notes, GitHub repos.
- AI finds starter resources per step from open sources, saves them as links with summaries, and downloads openly licensed PDFs into the shared library.
- A Practice section with links to mock interviews, question banks and coding practice.

Collaboration and changes

- Comments and discussion on each step.
- Suggestions: any member can leave a suggestion on another member's step or phase; the owner accepts or dismisses it.
- "Since you were away" screen on return: the master's changes, suggestions for you, newly finished dependencies.
- Notifications for deadlines, suggestions, approvals and unblocked steps.

Tracking

- Progress is saved continuously and works offline.

- Dashboard: percent done per phase and per member, on-track or behind for the deadline.
- Active-time tracking with planned vs actual hours, including overtime.
- Contribution record: who finished what and how long it took.

GitHub

- Link a GitHub account and attach repos to a classroom.
- Steps can link to issues or branches; a commit message such as `closes step 12` marks that step done.
- Commit activity per member shown on the dashboard.

General

- Light and dark themes; works on phone, tablet, web and desktop layouts.
- Export a classroom report as PDF.

Flowspace (node graph view)

Flowspace, the graph page, is a custom-built Flutter canvas, not an off-the-shelf package, because no existing package reaches the quality this app depends on.

What the user sees

- **Three zoom levels:**
  - Far: phases as large cards with a progress ring each.
  - Middle: steps as nodes with owner avatar, status colour and due date.
  - Close: the full step card inline, with files, comments, suggestions and checklist.
- **Member lanes:** each member's steps sit in their own horizontal lane; phases run left to right as columns.
- **Dependency arrows** between steps, including across lanes, coloured by status: grey waiting, blue in progress, green done, red blocked.
- **Critical path:** the chain of steps that decides the deadline is highlighted; delays on it are flagged.
- **Focus on me:** dims everyone else's work and centres the user's next available step.
- **Graph or timeline:** one toggle shows the same data as a timeline (Gantt) with dates to scale.
- **Filters:** by member, phase, status, overdue.
- **Minimap** in a corner for large plans; a "fit to screen" button.

- **Feedback:** a short animation when a step completes, and the arrows it unblocks light up.
- **Badges** on nodes for new suggestions, comments and master changes since the last visit.

Interactions

Action	Phone and tablet	Web and desktop
Move around	Drag with one finger	Drag, scroll wheel, arrow keys
Zoom	Pinch	Ctrl + scroll, + and − keys
Open a step	Tap	Click
Edit or menu	Long-press	Right-click
Add a dependency	Drag from a node's handle to another node	Same, plus a link mode shortcut
Reorder or move a step	Long-press and drag	Drag
Mark done	Swipe on the step card	Checkbox or keyboard shortcut

How it's built

- **Layout:** a layered (Sugiyama-style) layout computes columns from dependencies, with lanes fixed per member, then minimises crossing arrows. Computed off the main thread so large graphs don't freeze.
- **Rendering:** `CustomPainter` inside `InteractiveViewer`; only on-screen nodes are drawn, so 500+ steps stay smooth.
- **Level of detail:** what each node shows depends on zoom, so text never becomes unreadable clutter.
- **Live updates:** Firestore streams update nodes in place without re-laying out the whole graph.
- **Rules:** creating a circular dependency is refused with a clear message.
- **Accessibility:** a list view of the same plan for screen readers and small screens.

Prototype first

Build the graph with fake data before any other feature, test it with 5 to 10 real users on phone and laptop, and only move on once it feels effortless.

Kanvas (personal workspace)

Every signed-in user gets Kanvas, a private infinite canvas for ideas, notes, diagrams and drafts, which can be turned into real work in a classroom.

What it holds

- Freehand drawing and shapes, with smooth pen support on tablets and styluses.
- Sticky notes, text blocks and checklists.
- Diagrams: boxes, arrows and connectors that snap to shapes.
- Images, PDFs, links and videos dropped onto the canvas as preview cards.
- Multiple boards, organised into folders, with search across all of them.

How it connects to the rest of Octria

- **Idea to task:** select a group of notes and turn them into a new classroom; the AI builds the roadmap from them.
- **Pin to a step:** attach a Kanvas board to any step in Flowspace so teammates can see the thinking behind it.
- **Share a board:** private by default; can be shared read-only or editable with a classroom or a person.
- **AI helpers:** tidy messy notes into a list, turn a sketch of boxes into a clean diagram, summarise a board.

How it's built

- Board contents are stored as JSON shapes in Firestore (`users/{uid}/kanvas/{boardId}`), synced in real time and cached offline.
- Images and files placed on a board go to the user's private Drive folder.
- Built on the same custom Flutter canvas engine as Flowspace (pan, zoom, rendering only what's on screen), so effort is shared between the two.
- Undo and redo, autosave, and version history per board.

Ofy (Ask, Show, Do)

Ofy is a voice-and-text AI helper that can see what you are doing in Octria, point at things on screen, guide you step by step, and act for you after you approve. It is modelled on the HeyTora demo, scaled to fit inside the app.

What it does inside Octria

- **Ask:** a floating mic button on every screen. Speak or type: "Why is my step blocked?", "What should I do next?", "Who is behind schedule?". Speech-to-text runs on the device, so it is free.
- **Show:** the AI highlights the exact node in Flowspace, the button or field it is talking about, with a circle or glow and a pointer.
- **Guide:** multi-step walkthroughs shown as cards ("Step 1 of 2: tap the API step. Step 2 of 2: drag a link to the UI step."), advancing as you complete each one.
- **Do:** performs actions such as creating steps, linking dependencies, reassigning work, marking done, adding a resource or tidying a Kanvas board. Every action shows an Approve / Deny card first, with an "always allow this action" option.

This is easier than the HeyTora version because Octria owns its own screens: the AI reads the app's data and layout directly instead of interpreting screenshots, so it is faster, cheaper and more accurate.

Help with work outside Octria

- **Screenshot help:** attach or paste a screenshot (an error, a spreadsheet, code); Gemini explains it and the app draws circles and arrows on the image to show the problem.
- **Screen share (web and desktop):** share a window with Octria; the assistant looks at it when you ask a question and answers with annotated snapshots.
- **Not included:** clicking or typing inside other apps on your computer. That needs a separate desktop agent with system-level permissions; it can be considered after launch for the Windows and macOS builds only.

How it's built

- Each screen exposes a list of named, addressable elements (nodes, buttons, fields), so the AI can return "highlight `step:12`" or "tap `button:add-dependency`".
- Gemini receives the user's question, the current screen's element list and the relevant classroom data, and returns JSON: an answer, highlights, guide steps and proposed actions.
- Actions run through the same Firestore permission rules as manual edits, so the assistant can never do what the user isn't allowed to.

- Free-tier limits: answers and screen context are kept small, and screenshots are only sent when the user attaches or shares one.

Mock tests and practice links

Each phase can end with a mock test built from that phase's own content, so people can check what they learned before moving on. It applies to any field, not just interviews or exams. Practice sites stay one tap away as links.

Mock tests

- **Mock exam:** written and multiple-choice questions, answered in one sitting.
- **Mock interview:** one question at a time with follow-ups, answered by typing or speaking. Speech-to-text runs on the device, as it does for Ofy.
- **Based on the phase:** questions come from that phase's steps, descriptions, checklists, skills and notes, not from outside it.
- **When it's offered:** once a phase's steps are done. People can take it earlier or skip it, and it never blocks the next phase.
- **Results:** a score, feedback on each answer, and links back to the steps worth revising. A retake uses new questions. Results are private by default, and each person can turn on sharing so everyone in the group can see theirs.

Difficulty

Levels are Easy, Medium and Hard.

Who	What they can set
Each person	Their own level for their own test
Master	One level for everyone in the classroom, which replaces personal choices until the master clears it

In an individual classroom the person is the master, so the choice is simply theirs.

Practice links

- Every classroom has a Practice section, reachable at any time, with links to mock interview sites, question banks and coding practice.
- The AI suggests links that fit the task type and each phase, and anyone can add their own.

- Links open the outside site. Octria stores no copied questions and has no code runner.

How it's built

- Gemini returns questions and grading as JSON, like the other AI pipelines.
- A test is cached per phase, format and difficulty, so a group shares one set of questions and uses fewer free-tier calls.
- Spoken answers are sent as text, never as audio.

Roles and permissions

The master controls everything; members control only their own steps, and every rule is enforced in Firestore Security Rules, not just hidden in the UI.

Action	Master	Co-master	Member
Edit task, deadline, classroom settings	Yes	No	No
Add, edit, delete phases	Yes	Yes	No
Add, edit, delete any member's steps	Yes	Yes	No
Add, edit, delete own steps	Yes	Yes	Yes
Mark own step done	Yes	Yes	Yes
Approve or reject a finished step	Yes	Yes	No
Leave a suggestion on anyone's step or phase	Yes	Yes	Yes
Accept or dismiss suggestions on own step	Yes	Yes	Yes
Comment on any step	Yes	Yes	Yes
Upload to shared library	Yes	Yes	Yes
Delete from shared library	Yes	Yes	Own uploads only
Invite or remove members, assign co-master	Yes	Invite only	No
Run AI re-plan for the whole classroom	Yes	Yes	No
Set mock test difficulty for everyone	Yes	No	No

Action	Master	Co-master	Member
Choose own mock test difficulty, when none is set for everyone	Yes	Yes	Yes
Share own mock test results with the group	Yes	Yes	Yes

Every change by a master or co-master to another person's step is written to the activity log, so it appears on that person's "Since you were away" screen.

Any member can hand one of their own steps to any other member; the handover is logged and the receiver is notified. Step approval by the master can be switched on or off per classroom.

Data model

Firestore holds everything except file contents; classrooms are top-level documents with their plan and activity in subcollections.

Collection	Key fields
`users/{uid}`	name, photo, bio, githubLogin, driveFolderId, createdAt
`users/{uid}/skills/{skillId}`	name, level (1–5), source (resume / manual / earned), firstSeen, lastUsed
`users/{uid}/growth/{date}`	skillCount, stepsDone, hoursActive (daily snapshot for the growth map)
`classrooms/{cid}`	title, description, taskType, mode (individual / group), masterUid, coMasterUids, deadline, hoursPerWeek, inviteCode, status, driveFolderId, mockDifficulty
`classrooms/{cid}/members/{uid}`	role, joinedAt, plannedHoursPerWeek, lastSeenAt, shareMockResults
`classrooms/{cid}/phases/{pid}`	title, order, description, status
`classrooms/{cid}/steps/{sid}`	phaseId, title, description, ownerUid, order, status (todo / doing / review / done), dependsOn[], estimateHours, startDate, dueDate, skills[], githubRef

Collection	Key fields
`classrooms/{cid}/steps/{sid}/comments/{id}`	authorUid, text, createdAt
`classrooms/{cid}/suggestions/{id}`	targetType (step / phase), targetId, fromUid, toUid, text, status (open / accepted / dismissed)
`classrooms/{cid}/resources/{rid}`	type (file / link / video / note / repo / practice), title, url or driveFileId, scope (shared / private), ownerUid, stepId, tags, summary
`classrooms/{cid}/activity/{id}`	actorUid, action, targetId, before, after, createdAt (feeds "Since you were away")
`classrooms/{cid}/timeLogs/{id}`	uid, stepId, start, end, activeMinutes
`classrooms/{cid}/mockTests/{tid}`	phaseId, format (exam / interview), difficulty (easy / medium / hard), questions[], createdAt
`classrooms/{cid}/mockAttempts/{id}`	uid, testId, answers[], score, feedback[], finishedAt

The "Since you were away" screen queries `activity` and `suggestions` for entries newer than the member's `lastSeenAt`, then updates it.

Storage

Files live in users' own Google Drive and everything else is a lightweight record in Firestore, which keeps storage free for you.

- **Shared library:** a Drive folder created in the master's Drive when the classroom is made, shared with every member. Members uploading to it write into that folder.
- **Private library:** a folder per member, in that member's own Drive. Only they can see it.
- **Permission scope:** `drive.file` only, so the app can touch only files it created or the user picked, never the rest of their Drive.
- **Links, videos, notes, repos:** no file stored; the app saves the URL, title, thumbnail and an AI summary, shown as a preview card.
- **Attach anywhere:** any resource can be pinned to a step or phase, and appears in that step's card.

- **Limits:** a per-upload size cap (for example 100 MB) and warnings when the master's Drive is nearly full, since shared files count against their quota.
- **When a member leaves:** their private files stay with them; shared files stay with the master.

If the master leaves or deletes their account, the classroom and its shared folder pass to the co-master.

AI pipelines

Every AI call runs in a Cloud Function, returns strict JSON that the app validates, and produces a draft the user edits before it is saved.

1. **Understand the task.** Input: title, description, any uploaded brief. The AI asks up to 5 short follow-ups (deadline, hours per week, current level, goal, team size), then classifies the task type.
2. **Pick a planning style by type:**
    - Build (app, website, hardware): modules, dependencies, integration and testing phases.
    - Exam or study: syllabus topics, learning, revision rounds, mock tests, spaced repetition.
    - Research: literature, method, experiments, writing, review.
    - Event or creative: milestones, deliverables, rehearsals or drafts.
    - Other: a general phases-and-steps plan.
3. **Generate the roadmap.** Output: phases, steps, dependencies, hour estimates, required skills per step. The app then lays dates onto a timetable from the deadline and hours per week, so dates are calculated, not guessed by the AI.
4. **Extract skills from resumes.** PDF text is extracted on the server, the AI returns a skill list with levels, and the user confirms it.
5. **Split work (groups).** Matches step skills to member skills, balances hours, avoids giving one person every blocking step. The master reviews the split.
6. **Find resources.** For each step, searches open sources (official docs, arXiv, YouTube, open textbooks), saves links with one-line summaries, and downloads openly licensed PDFs to the shared library.
7. **Re-plan.** Triggered by the master: re-estimates remaining work, re-assigns orphaned steps, keeps finished work untouched.
8. **Create and grade mock tests.** Input: one phase's steps, descriptions, checklists, skills and notes, plus the format and difficulty. Output: questions, then a score and feedback on each answer.

Guardrails:

- A JSON schema check on every response; on failure, retry once, then show an error.
- Cache results per classroom; only regenerate on request.
- Daily AI call limit per user so the free tier isn't exhausted.
- Never send another member's private data in a prompt.

Tracking and growth

Active time, not logged-in time, is what gets recorded, and it feeds both the hours comparison and each user's growth map.

Active-time tracking

- A timer runs while a step is open and the app is in the foreground.
- It pauses after 5 minutes with no input, and when the app is backgrounded or the browser tab is hidden.
- Members can also start and stop a timer on a step by hand, and correct an entry.
- Each session is saved as a time log tied to a step.

Planned vs actual

- Each member's planned hours per week are set when they join.
- The dashboard shows planned vs actual per week, per step and per phase.
- Hours beyond the plan are recorded and shown as overtime.
- Visible to every member of the classroom.

Growth map

- Skill radar: current skills and levels.
- Timeline: skill count and levels over time, from daily snapshots.
- Finishing a step adds experience to the skills it required; enough experience raises a skill's level.
- New skills used in a step for the first time are added as "earned".
- Totals on the profile: classrooms finished, steps done, hours worked.

Staying free

The app can run at zero cost for a small-to-medium user base if the expensive parts are capped from day one.

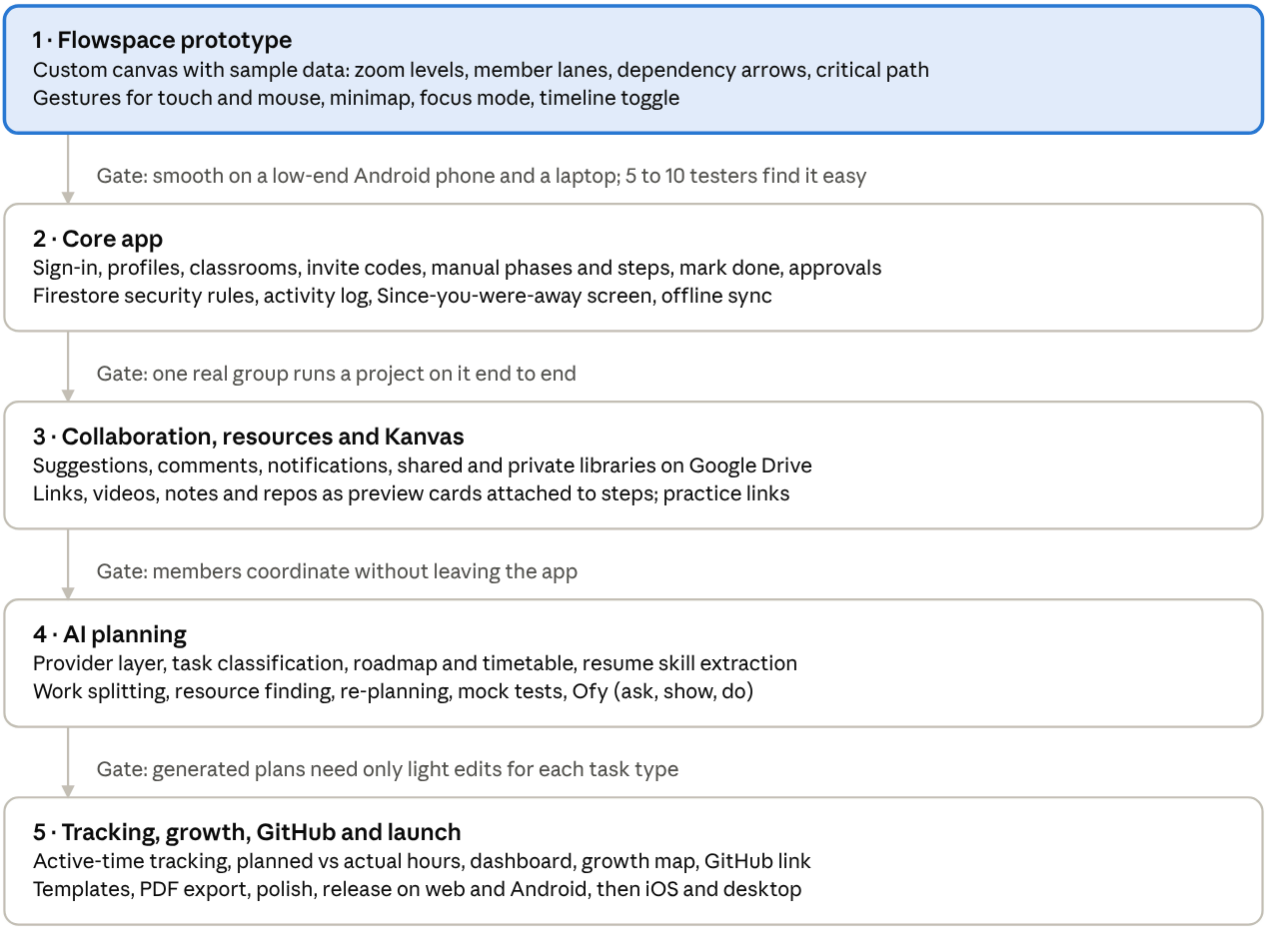
Cost source	How it stays free
Firebase plan	Blaze (pay-as-you-go) is needed for Cloud Functions; it keeps a free monthly quota. Set budget alerts at a small amount.
Firestore reads	Paginate the activity feed; listen only to the open classroom; rely on the offline cache.
File storage	Users' own Google Drive, not Firebase Storage.
AI calls	Gemini free tier, cached results, daily limit per user.
Push notifications	Firebase Cloud Messaging is free.
Web hosting	Firebase Hosting free quota.
App stores	Google Play has a one-time fee; Apple's developer program has a yearly fee. Web and Android can launch first.

If the app grows past the free quotas, options are a donation link, a paid "pro" tier for heavy AI use, or letting users add their own API key.

Build phases

The graph is built and tested first, because if it doesn't feel great nothing else matters; AI comes after the core app works by hand.

Graph first: each phase starts only after the one before passes its gate



build phases · 5 phases, 4 gates

Each gate is a check with real users before the next phase starts, so problems surface while they're still cheap to fix.

Risks and open questions

- **Graph performance on low-end phones and Flutter web.** Mitigation: prototype first and test on a cheap Android phone.
- **AI plan quality varies by task type.** Mitigation: editable drafts and per-type prompts.
- **Gemini free-tier limits or terms change.** Mitigation: the AI provider layer makes switching one function.
- **Google OAuth verification.** Public apps using Drive need Google's app review; `drive.file` is the lightest scope to get approved.
- **Privacy of resumes and time tracking.** Needs a clear privacy policy and a way to delete all personal data.
- **AI grading can be wrong.** Mitigation: show the feedback behind each mark, and never let a score block work.

Decisions

Answered on 3 Nov 2026; visual design waits until the design skills are added.

Question	Decision
App name	Octria (previously Modus); the graph page is called Flowspace; the AI is Ofy
AI provider	Gemini API free tier
Hours visibility	All members see each other's tracked hours
Master leaves	Classroom passes to the co-master
Step approval	Master approval can be turned on or off per classroom
Group size	Up to 10 members
Idle timeout	5 minutes
Resumes and Gemini	Sending resumes through the free tier is acceptable
Resource finding	Save links and download openly licensed PDFs
Reassigning work	A member can hand their own step to any member
Launch order	Web and Android first, then iOS and desktop
Languages	English only at launch
First users	Abhin tests first, then a team of friends
Visual design	Pending: to be set after adding the Apple Design and Impeccable skills and Playwright MCP
Mock test difficulty	Each person sets their own; the master can set one level for everyone
Mock test results	Private by default; each person can turn on sharing with the whole group
Mock test rules	Exam or interview format; Easy, Medium or Hard; optional after each phase; never blocks the next phase
Practice sites	Links only; no stored questions and no code runner